

AI ENGINEER INTERNSHIP

Technical Case Study

Intelligent Document Extraction, Validation & API Platform

Objective

Build and deploy an end-to-end AI-powered document extraction and validation platform that accepts supported financial documents through a frontend and REST APIs, extracts all available fields accurately, performs document and financial validations, stores processed results, and returns consistent structured JSON.

Sample Documents are Provided to the participants for Development and testing.

Parameter	Requirement
Duration	3 calendar days
Role	AI Engineer Intern
Submission	Public GitHub repository link + frontend + deployed API + documentation
AI assistants	Permitted (ChatGPT, Claude, Gemini, Copilot, Cursor, etc.)
Assessment focus	Working deployed solution, extraction accuracy, API/application quality, validation, engineering discipline and candidate understanding

Important

A locally running application alone is not sufficient. Deployment is mandatory. Evaluators must be able to access the submitted frontend and API without running the candidate's code locally.

Solution Presentation (PPT) is Mandatory

1. Problem Statement

Financial-services teams frequently receive invoices and financial statements in native or scanned PDF / JPG / PNG format. Different layouts, image quality, OCR errors and varying financial line items make manual extraction slow and inconsistent. The organisation wants a reusable Document Intelligence service that can extract the complete information present in a document, validate key financial relationships and make processed results available through a deployed dashboard and REST APIs.

Your task is to design, build and deploy a working prototype that performs the complete flow below:

```
Document Upload
  ↓
Document Validation
(PDF / JPG / PNG / file integrity / page limit)
  ↓
Text Extraction / OCR
  ↓
AI-based Field & Table Extraction
  ↓
Structured JSON Output
  ↓
Financial Calculation Validation
  ↓
Confidence / Evidence
  ↓
Store Processing Result
  ↓
PASS / FAILED
  ↓
Dashboard + REST API Response
```

2. Documents in Scope

Document Type	Extraction Requirement
Invoice	Extract all visible invoice fields and line-item/table values. At minimum: invoice_number, invoice_date, vendor_name, customer_name, currency, subtotal, tax_amount, discount (if present), total_amount.
Balance Sheet	Extract all statement header information, periods, currency and every visible financial line item/value. At minimum capture total_assets, total_liabilities and total_equity where present.
Profit & Loss	Extract all statement header information, periods, currency and every visible income/expense line item/value. At minimum capture revenue, cost_of_sales/COGS, gross_profit, operating_expenses, operating_profit, tax and net_profit where present.
Cash Flow Statement	Extract all statement header information, periods, currency and every visible cash-flow line item/value. At minimum capture operating_cash_flow, investing_cash_flow, financing_cash_flow, opening_cash, net_change_in_cash and closing_cash where present.

Only these four document categories are in scope. Automated document schema-selection logic are NOT required for this case study. The document type may be selected in the frontend and sent as request metadata to the API. The assessment focuses on extraction completeness, accuracy, validation, API design, storage and deployment.

3. Input Requirements

- Support PDF / JPG / PNG documents only. Unsupported uploads must be identified and rejected as unsupported.
- Support both native PDFs and scanned/image-based formats using text extraction and/or OCR as required.
- Support documents up to 3 pages.
- Validate file type, empty/corrupted files, page count and basic file integrity before extraction.
- Document validation is the input-control layer; it is not document-type classification.

4. Mandatory Functional Requirements

4.1 Document Validation

Before OCR or AI extraction, validate the uploaded file. A valid submission must be a readable PDF / JPG / PNG of no more than 3 pages. Unsupported formats, empty files, corrupted PDFs or documents exceeding the page limit must fail gracefully with a clear error response.

```
{
  "file_name": "sample_invoice.pdf",
  "file_validation": {
    "file_type": "application/pdf",
    "is_supported": true,
    "is_readable": true,
    "page_count": 1,
    "status": "PASS"
  }
}
```

4.2 Complete Field & Table Extraction

Extract ALL meaningful information visible in the document, not only a small fixed list of fields. This includes header fields, dates, parties, currencies, totals, financial statement line items, comparative-period values, and invoice/statement tables where present. Values must accurately reflect the source document. Missing values must be returned as null; the system must not invent or infer a value that is not supported by the document.

4.3 Evidence / Grounding

For extracted values, provide supporting source text where practical. Page number is strongly preferred. Evidence must allow the evaluator to trace important extracted values back to the source document. A confidence score is OPTIONAL.

```
{
  "field": "total_amount",
  "value": 13125.00,
  "confidence": 0.97,           // OPTIONAL
  "evidence": {
    "source_text": "Total Amount Due: USD 13,125.00",
    "page_number": 1
  }
}
```

4.4 Validation

Document	Minimum Financial Validation
----------	------------------------------

Invoice	Where line items are available: $\text{Quantity} \times \text{Unit Price} \approx \text{Line Total}$. Sum of all line totals should reconcile to the reported subtotal/total where applicable. Where tax is separately shown: $\text{Taxable Amount} + \text{Tax} \approx \text{Total}$. Where cash and change are available: $\text{Cash Paid} - \text{Total Amount} \approx \text{Change}$. Candidates must handle cases where GST/tax is already included in the displayed total.
Balance Sheet	$\text{Total Capital \& Liabilities} \approx \text{Total Assets}$. Where sufficient line items are available, the sum of individual Capital & Liability components should reconcile to the reported Capital & Liabilities total, and the sum of individual Asset components should reconcile to the reported Assets total. Perform the validation independently for each year/period present in the document.
Profit & Loss	Validate: $\text{Interest Earned} + \text{Other Income} \approx \text{Total Income}$; $\text{Interest Expended} + \text{Operating Expenses} + \text{Provisions \& Contingencies} \approx \text{Total Expenditure}$; $\text{Total Income} - \text{Total Expenditure} \approx \text{Consolidated Net Profit before Minority Interest}$; $\text{Profit before Minority Interest} - \text{Minority Interest} \approx \text{Consolidated Net Profit attributable to the Group}$. Where appropriations are present: $\text{Current Profit} + \text{Brought Forward Profit} \approx \text{Total Available for Appropriation}$. Perform validations for each comparative year/period present.
Cash Flow Statement	Validate: $\text{Net Cash Flow from Operating Activities} + \text{Net Cash Flow from Investing Activities} + \text{Net Cash Flow from Financing Activities} + \text{FX / Translation Adjustment} \approx \text{Net Increase in Cash \& Cash Equivalents}$. Then validate: $\text{Opening Cash \& Cash Equivalents} + \text{Net Increase in Cash} + \text{Cash Acquired on Amalgamation/Other Applicable Adjustments} \approx \text{Closing Cash \& Cash Equivalents}$. Parentheses/bracketed values must be treated as negative values. Validate each period independently where comparative data is available.

Important: Financial calculations must be based on the actual fields present in the source document. If a field required for a particular validation is not present, the validation should return NOT_APPLICABLE rather than assume or invent a value.

The validation result must be included in the structured JSON and should contain the **formula/check performed, input values used, calculated value, reported value, variance and PASS / FAIL / NOT_APPLICABLE status**.

4.5 Processing Status

Status	Meaning
PASS	Required fields are extracted accurately and required validations pass.
FAILED	Document could not be processed, is invalid, corrupted or unsupported.

5. Mandatory REST API

The deployed backend must expose REST APIs. Swagger/OpenAPI documentation must be available on the deployed environment.

Method	Endpoint	Purpose
POST	/api/v1/documents/process	Upload and process a PDF / JPG / PNG document.
GET	/api/v1/documents/{document_name}	Retrieve the latest structured result using the document/file name.

GET	/api/v1/documents	List processed documents for the frontend dashboard.
GET	/api/v1/health	Health check for the deployed service.

5.1 POST Request

```
POST /api/v1/documents/process
Content-Type: multipart/form-data

file: <document.pdf>
document_type: invoice | balance_sheet | profit_and_loss | cash_flow_statement
```

The API may process synchronously for this assignment. The processed result must be stored so that it appears in the frontend dashboard and can later be retrieved by document name. If the same document name is processed more than once, the GET-by-name endpoint should return the latest result; retaining prior versions is optional.

5.2 Mandatory Structured Response

```
{
  "document_name": "sample_invoice.pdf",
  "document_type": "invoice",
  "processing_status": "PASS",
  "overall_confidence": 0.96,    // OPTIONAL
  "file_validation": {
    "file_type": "application/pdf",
    "is_supported": true,
    "is_readable": true,
    "page_count": 1,
    "status": "PASS"
  },
  "extracted_data": {
    "invoice_number": {"value": "INV-23891", "confidence": 0.99,    // OPTIONAL "page_number": 1},
    "invoice_date": {"value": "2026-08-15", "confidence": 0.98,    // OPTIONAL "page_number": 1},
    "vendor_name": {"value": "ABC Technologies", "confidence": 0.97, // OPTIONAL "page_number": 1},
    "currency": {"value": "USD", "confidence": 0.99,    // OPTIONAL "page_number": 1},
    "subtotal": {"value": 12500.00, "confidence": 0.98,    // OPTIONAL "page_number": 1},
    "tax_amount": {"value": 625.00, "confidence": 0.97,    // OPTIONAL "page_number": 1},
    "discount": {"value": 0.00, "confidence": 0.95,    // OPTIONAL "page_number": 1},
    "total_amount": {"value": 13125.00, "confidence": 0.99,    // OPTIONAL "page_number": 1},
    "line_items": [
      {"description": "Service A", "quantity": 1, "unit_price": 12500.00, "amount": 12500.00}
    ]
  },
  "validation": {
    "checks": [
      {
        "name": "invoice_total_check",
        "formula": "subtotal + tax_amount - discount",
        "operands": {"subtotal": 12500.00, "tax_amount": 625.00, "discount": 0.00},
        "calculated_value": 13125.00,
        "reported_value": 13125.00,
        "variance": 0.00,
        "status": "PASS"
      }
    ]
  }
}
```

```

    "overall_status": "PASS",
    "issues": []
  },
  "processing_metadata": {
    "ocr_used": true,
    "processed_at": "2026-09-08T12:00:00Z",
    "processing_time_ms": 2840
  }
}

```

The exact internal implementation may differ, but the response must be valid JSON and consistent across requests. It must include document name, supplied document type, file validation, complete extracted data as clear key-value pairs, financial calculation validations, processing status and processing metadata. Confidence scores are optional. If provided, they should be meaningful and explainable rather than arbitrary LLM-generated values.

5.3 Error Response

```

{
  "error": {
    "code": "UNSUPPORTED_FILE_TYPE",
    "message": "Only PDF / JPG / PNG documents are supported."
  }
}

```

6. Mandatory Frontend

A deployed frontend must be provided using HTML/CSS (JavaScript may be used where required) and connected to the deployed Python API/application. FastAPI, Flask or another suitable Python web/API framework may be used. A separate React, Node.js or similar frontend stack is not required.

- Document type selection, PDF / JPG / PNG upload control and Process action.
- Dashboard showing all processed documents with document name, type, status and processed time. Confidence may be displayed if implemented.
- Allow the user to open a processed document result from the dashboard and view extracted fields/tables.
- Display extracted key-value pairs, tables/line items and financial validation/calculation results in a clear and readable manner.
- Clearly highlight missing/unreadable fields and failed financial validations. If confidence is implemented, low-confidence values should also be highlighted.
- Provide a way to view the raw structured JSON returned by the backend.
- Frontend must call the deployed backend/API. The processed-document dashboard must be connected to a database or persistent data store (for example SQLite, PostgreSQL, MySQL or an equivalent option) so processed results remain available during evaluation.

7. Mandatory Deployment

Deployment is a submission requirement

Both frontend and backend/API must be accessible to evaluators through live URLs at the time of evaluation. For Deployment use Platforms like Render, Railway, Koyeb etc.. A localhost-only submission will be considered incomplete.

Requirement	Expected Submission
Frontend	Publicly accessible URL
Backend API	Publicly accessible base URL
API Documentation	Swagger/OpenAPI URL (for example /docs)
Health Endpoint	Working /api/v1/health (Must)
Repository	Public GitHub repository URL containing the complete submitted source code
Configuration	No secrets/API keys committed to source code

Candidates may use any reasonable free/free-tier deployment platform available to them. Paid infrastructure is not expected. The final code must be pushed to a PUBLIC GitHub repository and the repository link must be submitted with the deployment URLs.

8. Required Code / Repository Structure

The following separation of concerns must be preserved. Equivalent naming is acceptable. Candidates should not place the complete application, OCR, extraction, API, database, validation, logging and exception-handling logic into a single file.

```

project-root/
├── backend/
│   ├── app/
│   │   ├── main.py
│   │   ├── api/
│   │   │   ├── routes/
│   │   │   └── documents.py
│   │   ├── core/
│   │   │   ├── config.py
│   │   │   ├── database.py
│   │   │   └── logging.py
│   │   ├── models/
│   │   │   └── document.py
│   │   ├── schemas/
│   │   │   ├── document.py
│   │   │   └── extraction.py
│   │   ├── services/
│   │   │   ├── document_validation_service.py
│   │   │   ├── ocr_service.py
│   │   │   ├── extraction_service.py
│   │   │   └── financial_validation_service.py

```

```

| | | └── document_service.py
| | └── repositories/
| | | └── document_repository.py
| | └── utils/
| └── tests/
| | └── test_api.py
| | └── test_validation.py
| | └── test_extraction.py
| └── requirements.txt OR pyproject.toml
|
└── frontend/
    ├── templates/
    |   ├── dashboard.html
    |   └── document_result.html
    ├── static/
    |   ├── css/
    |   └── js/
    └── <frontend application files>
|
└── docs/
    ├── architecture.png OR architecture.pdf
    └── solution_presentation.pdf
|
└── sample_outputs/
    └── <JSON results>
|
└── .env.example
└── .gitignore
└── README.md

```

Code-quality rule

API keys, credentials and tokens must be read from environment variables. Best coding practices such as validation, logging, exception handling, modularity and secure configuration carry high weight in the evaluation.

9. Technical Expectations

- Return all extracted fields as consistent, machine-readable key-value pairs using structured outputs (for example Pydantic/JSON Schema or equivalent). Financial tables and invoice line items should use structured arrays/objects.
- Separate file validation, OCR/parsing, extraction, financial validation, persistence/database access and API concerns.

- Implement proper exception handling for invalid files, OCR failures, model/API failures, timeouts, database failures and unexpected processing errors. The application must fail gracefully without exposing stack traces or secrets to end users.
- Implement meaningful logging for request processing, validation failures, OCR/model calls, exceptions and major processing stages. Logs should be sufficient to diagnose failures during evaluation.
- Return appropriate HTTP status codes and consistent error responses.
- Do not hardcode document outputs, expected answers or validation outcomes.
- Extraction accuracy and completeness carry high weight. ALL meaningful visible fields, values, financial line items and table values should be extracted and returned as proper key-value pairs/structured arrays. Values will be checked against the source documents. Do not hallucinate unreadable or missing values.
- Participants are encouraged to use free or free-tier document/OCR services where useful, such as OCR.Space, Google Cloud Vision / Document AI, LlamaParse, or equivalent services. Open-source/local options such as Tesseract, PaddleOCR and EasyOCR are also permitted. Provider free-tier limits and availability may vary, so candidates should select an option appropriate for the 3-day assessment.
- Any LLM/model available to the participant can be used. Confidence scoring is OPTIONAL. Implementing a sensible, explainable confidence mechanism adds value to the submission, but candidates will not be penalized for omitting it.

10. Non-Functional Requirements

Area	Minimum Expectation
Reliability	Malformed/unsupported documents or downstream failures must not crash the service; return controlled errors.
Security	No embedded secrets; validate uploads; avoid unsafe file handling.
Maintainability	Modular code, clear responsibilities, reusable services and clean configuration.
Observability	Meaningful application logs, error logs and processing metadata sufficient for troubleshooting.
Usability	Dashboard is intuitive; processed documents are searchable/viewable and results are easy to understand.
Performance	Reasonable response time for documents up to 3 pages; document known bottlenecks.
Persistence	Processed-document metadata and extraction results must be stored in a database/persistent store and surfaced in the dashboard.

11. Testing Requirements

- Demonstrate processing for all four supported document types.
- Include at least one scanned/image-based PDF / JPG / PNG.
- Include at least one financial validation-failure, missing-field or unreadable-value scenario. If confidence scoring is implemented, a low-confidence scenario may also be demonstrated.
- Include at least one unsupported/invalid file scenario.
- Provide sample JSON outputs for processed test documents, including validation calculations.
- At least basic automated tests for file validation, financial calculations and one API flow are expected.

12. Candidate Deliverables

#	Deliverable	Mandatory
---	-------------	-----------

1	Public GitHub repository URL	Yes
2	Live deployed frontend URL	Yes
3	Live backend API URL	Yes
4	Swagger/OpenAPI documentation URL	Yes
5	README with setup, architecture, API, database/persistence, assumptions and deployment details	Yes
6	Architecture diagram	Yes
7	Sample JSON outputs	Yes
8	AI/tool usage declaration	Yes
9	Known limitations and production-improvement notes	Yes
10	Short demo / walkthrough during interview	Yes

13. README Must Include

- Solution overview and architecture.
- Technology stack and reason for major choices.
- Local setup instructions.
- Environment-variable list in .env.example (without real secrets).
- Deployed Application URL, backend API URL, Swagger/OpenAPI URL and public GitHub repository URL.
- API request examples, including POST upload, GET by document name and dashboard/list API.
- OCR/document parsing service and LLM/model used, including any free/free-tier service selected.
- Confidence calculation/interpretation if confidence scoring is implemented (optional).
- Financial validation rules and numerical tolerance implemented.
- Database/persistence approach and how processed documents are stored and retrieved.
- Known limitations.
- What you would change for a production deployment.
- AI coding assistants used and where they were used.

14. Permitted Use of Generative AI

Use of ChatGPT, Claude, Gemini, GitHub Copilot, Cursor and similar tools is permitted. The assessment will focus on the submitted solution and the candidate's ability to explain, debug and modify it during discussion.

Interview expectation

Be prepared to explain the architecture, prompts/model choices, OCR approach, validation logic, API design, deployment and any code submitted. Candidates may be asked to make a small change or diagnose an issue during the interview.

15. Evaluation Criteria

Area	Weight
End-to-end working solution & deployment	15%
Extraction completeness & accuracy	25%
API implementation & quality	15%
Frontend application, dashboard & database integration	10%
Financial validation & calculation correctness	10%
Code quality, logging, exception handling & security	15%
Testing, documentation & public GitHub quality	5%
Candidate understanding / technical discussion	5%

API evaluation includes correct request/response design, multipart upload, GET by document name, processed-document list API, valid JSON, HTTP status codes, input validation, persistence and controlled error responses.

Application evaluation includes: deployed HTML/CSS frontend, working document upload/process flow, database-backed processed-document dashboard, clear extracted key-value display, financial validation results, raw JSON view, usability and correct backend integration.

Total: 100%

16. Submission Checklist

- ☐ Solution Presentation (PPT) Submitted.
- Frontend URL opens successfully.
- API health endpoint returns success.
- POST document API accepts multipart PDF / JPG / PNG upload.
- GET-by-document-name API returns the latest stored processing result.
- Processed-document list API returns records used by the dashboard.
- JSON is valid, consistent and returns all extracted fields as proper key-value pairs / structured arrays.
- All 4 mandatory document types are supported.
- Scanned/image-based PDF / JPG / PNGs are handled successfully.
- Financial calculations are returned inside the JSON validation section.
- Processed-document dashboard is connected to a working database/persistent store.
- Proper input validation, logging and exception handling are implemented.
- No secrets/API keys are committed to the public GitHub repository.
- README, architecture diagram, deployment URLs and public GitHub repository link are included.
- Candidate can explain the submitted solution.

17. Completion Standard

Minimum successful submission

A deployed API that validates PDF / JPG / PNG inputs, processes all four supported document types, extracts all meaningful fields and table/line-item values accurately as structured key-value data, performs required financial calculations inside the JSON validation section, stores processed results in a database/persistent store, retrieves results by document name, and displays all processed documents through a working dashboard. The complete

source code must be available in a public GitHub repository.

Your goal is not to build a production-ready enterprise platform in three days. The goal is to demonstrate practical AI engineering: turning OCR/LLM capabilities into a reliable, structured, testable and deployable application.